In [16]:
```
import pandas as pd
import numpy as np

# Create dataset
data = {
    'Student_ID': range(1, 21),
    'Math_Score': [78, 85, np.nan, 90, 120, 67, 45, 89, 76, 88, 92, np.nan, 55,
    'English_Score': [65, 70, 75, np.nan, 85, 60, 55, 78, 82, 90, 88, 76, 54, np
    'Science_Score': [70, 88, 92, 85, 95, 60, np.nan, 80, 78, 84, 89, 77, 58, 65
    'Study_Hours': [2, 3, 4, 5, 10, 2, 1, 3, 4, 6, 7, 3, 2, 4, 5, 6, 8, 1, 9, 3]
}

df = pd.DataFrame(data)
df.head()
```

Out[16]:

	Student_ID	Math_Score	English_Score	Science_Score	Study_Hours
0	1	78.0	65.0	70.0	2
1	2	85.0	70.0	88.0	3
2	3	NaN	75.0	92.0	4
3	4	90.0	NaN	85.0	5
4	5	120.0	85.0	95.0	10

1. Handling Missing Values & Inconsistencies

In [15]:
```
df.isnull().sum()
```

Out[15]:
```
Student_ID       0
Math_Score       0
English_Score    0
Science_Score    0
Study_Hours      0
dtype: int64
```

In [4]:
```
df.notnull().sum()
```

Out[4]:
```
Student_ID       20
Math_Score       18
English_Score    18
Science_Score    19
Study_Hours      20
dtype: int64
```

Handle Missing Values:

In [5]:
```
# Replace missing with median
df['Math_Score'].fillna(df['Math_Score'].median(), inplace=True)
df['English_Score'].fillna(df['English_Score'].median(), inplace=True)
df['Science_Score'].fillna(df['Science_Score'].median(), inplace=True)
```

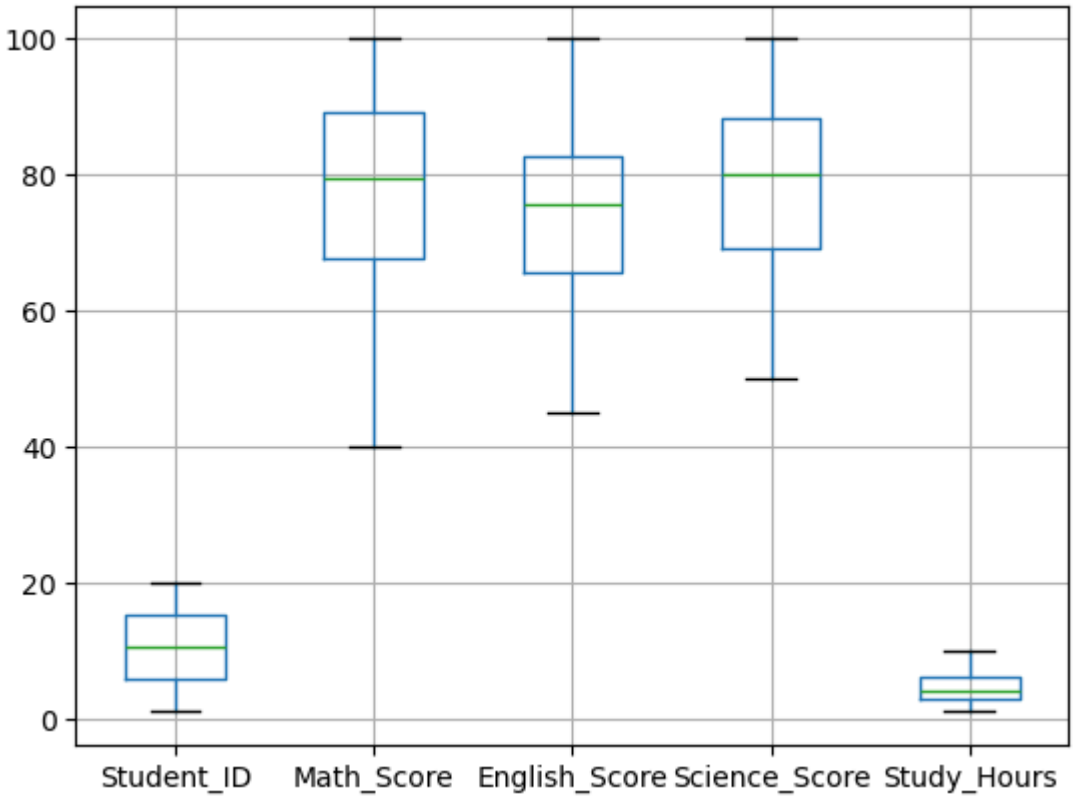
```
C:\Users\shrey\AppData\Local\Temp\ipykernel_27176\2502517521.py:2: FutureWarning:
A value is trying to be set on a copy of a DataFrame or Series through chained as
signment using an inplace method.
The behavior will change in pandas 3.0. This inplace method will never work becau
se the intermediate object on which we are setting values always behaves as a cop
y.

For example, when doing 'df[col].method(value, inplace=True)', try using 'df.meth
od({col: value}, inplace=True)' or df[col] = df[col].method(value) instead, to pe
rform the operation inplace on the original object.


  df['Math_Score'].fillna(df['Math_Score'].median(), inplace=True)
C:\Users\shrey\AppData\Local\Temp\ipykernel_27176\2502517521.py:3: FutureWarning:
A value is trying to be set on a copy of a DataFrame or Series through chained as
signment using an inplace method.
The behavior will change in pandas 3.0. This inplace method will never work becau
se the intermediate object on which we are setting values always behaves as a cop
y.

For example, when doing 'df[col].method(value, inplace=True)', try using 'df.meth
od({col: value}, inplace=True)' or df[col] = df[col].method(value) instead, to pe
rform the operation inplace on the original object.


  df['English_Score'].fillna(df['English_Score'].median(), inplace=True)
C:\Users\shrey\AppData\Local\Temp\ipykernel_27176\2502517521.py:4: FutureWarning:
A value is trying to be set on a copy of a DataFrame or Series through chained as
signment using an inplace method.
The behavior will change in pandas 3.0. This inplace method will never work becau
se the intermediate object on which we are setting values always behaves as a cop
y.

For example, when doing 'df[col].method(value, inplace=True)', try using 'df.meth
od({col: value}, inplace=True)' or df[col] = df[col].method(value) instead, to pe
rform the operation inplace on the original object.


  df['Science_Score'].fillna(df['Science_Score'].median(), inplace=True)
```

In [6]:
```
print(df)
```

	Student_ID	Math_Score	English_Score	Science_Score	Study_Hours
0	1	78.0	65.0	70.0	2
1	2	85.0	70.0	88.0	3
2	3	79.5	75.0	92.0	4
3	4	90.0	75.5	85.0	5
4	5	120.0	85.0	95.0	10
5	6	67.0	60.0	60.0	2
6	7	45.0	55.0	80.0	1
7	8	89.0	78.0	80.0	3
8	9	76.0	82.0	78.0	4
9	10	88.0	90.0	84.0	6
10	11	92.0	88.0	89.0	7
11	12	79.5	76.0	77.0	3
12	13	55.0	54.0	58.0	2
13	14	60.0	75.5	65.0	4
14	15	72.0	69.0	74.0	5
15	16	81.0	80.0	82.0	6
16	17	95.0	91.0	93.0	8
17	18	40.0	45.0	50.0	1
18	19	110.0	100.0	105.0	9
19	20	68.0	66.0	67.0	3

In [13]:
```
df.boxplot()
```

Out[13]: <Axes: >



Handle Inconsistencies (Invalid Values)

In [7]:
```
#Marks > 100 are invalid

df.loc[df['Math_Score'] > 100, 'Math_Score'] = 100
df.loc[df['Science_Score'] > 100, 'Science_Score'] = 100
```

In [8]:
```
df
```

Out[8]:

	Student_ID	Math_Score	English_Score	Science_Score	Study_Hours
0	1	78.0	65.0	70.0	2
1	2	85.0	70.0	88.0	3
2	3	79.5	75.0	92.0	4
3	4	90.0	75.5	85.0	5
4	5	100.0	85.0	95.0	10
5	6	67.0	60.0	60.0	2
6	7	45.0	55.0	80.0	1
7	8	89.0	78.0	80.0	3
8	9	76.0	82.0	78.0	4
9	10	88.0	90.0	84.0	6
10	11	92.0	88.0	89.0	7
11	12	79.5	76.0	77.0	3
12	13	55.0	54.0	58.0	2
13	14	60.0	75.5	65.0	4
14	15	72.0	69.0	74.0	5
15	16	81.0	80.0	82.0	6
16	17	95.0	91.0	93.0	8
17	18	40.0	45.0	50.0	1
18	19	100.0	100.0	100.0	9
19	20	68.0	66.0	67.0	3

2. Detect and Handle Outliers

Using IQR Method:

In [9]:

```
Q1 = df['Study_Hours'].quantile(0.25)
Q3 = df['Study_Hours'].quantile(0.75)
IQR = Q3 - Q1

lower = Q1 - 1.5 * IQR
upper = Q3 + 1.5 * IQR

# Detect outliers
outliers = df[(df['Study_Hours'] < lower) | (df['Study_Hours'] > upper)]
print(outliers)
```

```
Empty DataFrame
Columns: [Student_ID, Math_Score, English_Score, Science_Score, Study_Hours]
Index: []
```

In [12]:
```
print("Lower:", lower)
print("Upper:", upper)
```

```
Lower: -2.125
Upper: 10.875
```

Remove Outliers:

In [10]:
```
df = df[(df['Study_Hours'] >= lower) & (df['Study_Hours'] <= upper)]
```

In [11]:
```
df
```

Out[11]:

	Student_ID	Math_Score	English_Score	Science_Score	Study_Hours
0	1	78.0	65.0	70.0	2
1	2	85.0	70.0	88.0	3
2	3	79.5	75.0	92.0	4
3	4	90.0	75.5	85.0	5
4	5	100.0	85.0	95.0	10
5	6	67.0	60.0	60.0	2
6	7	45.0	55.0	80.0	1
7	8	89.0	78.0	80.0	3
8	9	76.0	82.0	78.0	4
9	10	88.0	90.0	84.0	6
10	11	92.0	88.0	89.0	7
11	12	79.5	76.0	77.0	3
12	13	55.0	54.0	58.0	2
13	14	60.0	75.5	65.0	4
14	15	72.0	69.0	74.0	5
15	16	81.0	80.0	82.0	6
16	17	95.0	91.0	93.0	8
17	18	40.0	45.0	50.0	1
18	19	100.0	100.0	100.0	9
19	20	68.0	66.0	67.0	3

3. Data Transformation

Example: Normalize Study Hours

In [19]:
```
df['Study_Hours'] = (df['Study_Hours'] - df['Study_Hours'].min()) / (df['Study_H
```

```
#Transformation helps in:

#Scaling data
#Reducing skewness
#Improving model performance
```

In [20]:
```
df
```

Out[20]:

	Student_ID	Math_Score	English_Score	Science_Score	Study_Hours
0	1	78.0	65.0	70.0	0.111111
1	2	85.0	70.0	88.0	0.222222
2	3	NaN	75.0	92.0	0.333333
3	4	90.0	NaN	85.0	0.444444
4	5	120.0	85.0	95.0	1.000000
5	6	67.0	60.0	60.0	0.111111
6	7	45.0	55.0	NaN	0.000000
7	8	89.0	78.0	80.0	0.222222
8	9	76.0	82.0	78.0	0.333333
9	10	88.0	90.0	84.0	0.555556
10	11	92.0	88.0	89.0	0.666667
11	12	NaN	76.0	77.0	0.222222
12	13	55.0	54.0	58.0	0.111111
13	14	60.0	NaN	65.0	0.333333
14	15	72.0	69.0	74.0	0.444444
15	16	81.0	80.0	82.0	0.555556
16	17	95.0	91.0	93.0	0.777778
17	18	40.0	45.0	50.0	0.000000
18	19	110.0	100.0	105.0	0.888889
19	20	68.0	66.0	67.0	0.222222

Log Transformation (reduce skewness)

In [23]:
```
df['Math_Score'] = np.log(df['Math_Score'])

#The function np.log() calculates the natural logarithm of each value

#It converts Math_Score values into logarithmic values.
```

In [24]:
```
df
```

Out[24]:

	Student_ID	Math_Score	English_Score	Science_Score	Study_Hours
0	1	1.471717	65.0	70.0	0.111111
1	2	1.491251	70.0	88.0	0.222222
2	3	NaN	75.0	92.0	0.333333
3	4	1.504035	NaN	85.0	0.444444
4	5	1.566007	85.0	95.0	1.000000
5	6	1.436201	60.0	60.0	0.111111
6	7	1.336753	55.0	NaN	0.000000
7	8	1.501549	78.0	80.0	0.222222
8	9	1.465737	82.0	78.0	0.333333
9	10	1.499028	90.0	84.0	0.555556
10	11	1.508908	88.0	89.0	0.666667
11	12	NaN	76.0	77.0	0.222222
12	13	1.388126	54.0	58.0	0.111111
13	14	1.409607	NaN	65.0	0.333333
14	15	1.453174	69.0	74.0	0.444444
15	16	1.480342	80.0	82.0	0.555556
16	17	1.515979	91.0	93.0	0.777778
17	18	1.305323	45.0	50.0	0.000000
18	19	1.547665	100.0	105.0	0.888889
19	20	1.439718	66.0	67.0	0.222222

In [ ]: